MODULE

18

Data, APIs & databases

Read files, call web APIs, store data in SQL, and serve your own REST API.

YOU WILL LEARN

Files, CSV & JSON	01_files_csv_json.py
Calling web APIs	02_http_requests.py
SQL with sqlite3	03_sqlite.py
Your own REST API	04_fastapi_app.py
Dates & time zones	05_dates_and_timezones.py
pickle vs JSON	06_pickle_vs_json.py
Text & bytes	07_text_and_bytes.py
Interview Q&A	INTERVIEW.md

WHY DATA

Every backend job is moving data around

THE PROBLEM

A typical ticket: **”Import the partner’s CSV every night, enrich it from their API, save it in our database, and expose it to the frontend.”**

THE IDEA

Four skills cover it: files (CSV/JSON), HTTP clients, SQL, and a REST API. This module does all four.

TRY IT FILES

CSV and JSON the professional way

```python
import csv, json
from pathlib import Path

orders = [{"id": 1, "item": "lamp", "price": 49.0},
          {"id": 2, "item": "cable", "price": 9.5}]

path = Path("orders.csv")
with path.open("w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["id", "item", "price"])
    writer.writeheader()
    writer.writerows(orders)

with path.open(encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
print(rows[0])                          # CSV gives you STRINGS
print(json.dumps(orders[0], indent=None))
print(sum(float(r["price"]) for r in rows))
```

OUTPUT

```
{'id': '1', 'item': 'lamp', 'price': '49.0'}
{"id": 1, "item": "lamp", "price": 49.0}
58.5
```

1. **pathlib.Path** instead of string paths.
2. Always pass **encoding=”utf-8”** (and **newline=””** for CSV).
3. CSV values are strings: convert them yourself.

PICTURE IT HTTP

What happens when you call an API

📮 HTTP is like sending a **letter with a form**: a method (what to do), an address (URL), headers (envelope info) and maybe a body. The reply has a **status code** and a body, usually JSON.



STEP BY STEP HTTP

Methods and status codes to know by heart

METHOD	MEANS	STATUS CODE	MEANS
GET	read (safe, no changes)	**200** OK / **201** Created	it worked / new resource
POST	create	**204** No Content	worked, nothing to return
PUT	replace (idempotent)	**400** / **422**	bad input / validation failed
PATCH	change some fields	**401** / **403**	not logged in / not allowed
DELETE	remove (idempotent)	**404** / **409**	not found / conflict
		429 / **500** / **503**	rate limited / our bug / overloaded

Idempotent = doing it twice has the same effect as once. Retries are safe only for idempotent calls.

COMMON MISTAKE HTTP CLIENTS

The fragile API call

✗ WRONG

```
import requests
r = requests.get(url)            # waits forever
data = r.json()                  # even on a 500
```

No timeout: one slow partner freezes your worker. No status check: you parse an error page as data.

✓ RIGHT

```
r = requests.get(url, params={"page": 2},
                 timeout=(3, 10))     # connect, read
r.raise_for_status()                  # 4xx/5xx -> error
data = r.json()
```

Always: a **timeout**, **raise_for_status()**, a **Session** for many calls, and retries with backoff.

TRY IT SQL

SQL with sqlite3: tables, joins, totals

```
import sqlite3

db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE users  (id INTEGER PRIMARY KEY, name TEXT);
CREATE TABLE orders (id INTEGER PRIMARY KEY, user_id INT, total REAL);
INSERT INTO users  VALUES (1, 'vishal'), (2, 'li'), (3, 'sam');
INSERT INTO orders VALUES (1, 1, 49), (2, 1, 20), (3, 2, 9.5);
""")
rows = db.execute("""
    SELECT u.name, COUNT(o.id) AS n, COALESCE(SUM(o.total), 0)
    FROM users u LEFT JOIN orders o ON o.user_id = u.id
    GROUP BY u.id ORDER BY 3 DESC
""").fetchall()
for r in rows:
    print(r)
```

OUTPUT

```
('vishal', 2, 69.0)
('li', 1, 9.5)
('sam', 0, 0)
```

1. **JOIN** connects tables by a key.
2. **LEFT JOIN** keeps users without orders (sam).
3. **GROUP BY** + COUNT/SUM = one row per user.

COMMON MISTAKE SQL

SQL injection

✗ WRONG

```
name = request.args["name"]
db.execute(f"SELECT * FROM users "
           f"WHERE name = '{name}'")
# name = "x' OR '1'='1"  -> every user!
```

User input becomes part of the SQL. An attacker can read or delete **everything**. Still in the OWASP Top 10.

✓ RIGHT

```
db.execute("SELECT * FROM users WHERE name = ?",
           (name,))        # a PARAMETER

# ORMs do this for you:
User.objects.filter(name=name)
```

Never build SQL with f-strings. Use **placeholders** (**?** or **%s**) or an ORM.

TRY IT SQL

Transactions: all or nothing

```
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE acc (name TEXT, bal INT CHECK (bal >= 0))")
db.executemany("INSERT INTO acc VALUES (?, ?)", [("vishal", 100), ("li", 0)])
db.commit()

def transfer(src, dst, amount):
    try:
        with db:                                  # commit or rollback
            db.execute("UPDATE acc SET bal = bal + ? WHERE name = ?", (amount, dst))
            db.execute("UPDATE acc SET bal = bal - ? WHERE name = ?", (amount, src))
    except sqlite3.IntegrityError:
        print(f"transfer of {amount} failed, rolled back")

transfer("vishal", "li", 30)
transfer("vishal", "li", 500)
print(db.execute("SELECT * FROM acc").fetchall())
```

OUTPUT

```
transfer of 500 failed, rolled back
[('vishal', 70), ('li', 30)]
```

1. **with db:** commits on success and rolls back on error.
2. The failed transfer left **no** half-done change.
3. This is the **A** in ACID: atomicity.

TRY IT REST API

Your own API with FastAPI

```
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel

app = FastAPI()
NOTES: dict[int, dict] = {}

class NoteIn(BaseModel):
    text: str

@app.post("/notes", status_code=201)
def create(note: NoteIn):
    nid = len(NOTES) + 1
    NOTES[nid] = {"id": nid, "text": note.text}
    return NOTES[nid]

@app.get("/notes/{nid}")
def read(nid: int):
    if nid not in NOTES:
        raise HTTPException(404, "note not found")
    return NOTES[nid]

c = TestClient(app)
print(c.post("/notes", json={"text": "buy milk"}).json())
print(c.get("/notes/9").status_code, c.get("/notes/9").json())
print(c.post("/notes", json={}).status_code)
```

OUTPUT

```
{'id': 1, 'text': 'buy milk'}
404 {'detail': 'note not found'}
422
```

1. Type hints = validation + docs at **/docs**.
2. Errors become proper status codes (404, 422).
3. Modules 21-28 go much deeper.

COMMON MISTAKE DATES AND TIME ZONES

Naive datetimes

✗ WRONG

```
from datetime import datetime
created = datetime.now()      # local, no zone
# server in Frankfurt, user in Mumbai,
# DST change in October... which hour?
```

A naive datetime has **no time zone**. Comparing or storing them across servers and DST changes gives wrong hours.

✓ RIGHT

```
from datetime import datetime, UTC
from zoneinfo import ZoneInfo
created = datetime.now(UTC)           # store UTC
local = created.astimezone(ZoneInfo("Asia/Kolkata"))
print(local.isoformat())              # show local
```

Rule: **store and compute in UTC**, convert to the user's zone only for display. Send ISO 8601 strings.

STEP BY STEP SERIALISATION

pickle vs JSON, text vs bytes

	USE IT FOR	WATCH OUT
JSON	APIs, config, anything shared	only dicts, lists, str, numbers, bool, None
pickle	Python-only caches, ML models you made	**loading untrusted pickle runs code**: never
str	text (Unicode characters)	`.encode("utf-8")` → bytes
bytes	files, network, images	`.decode("utf-8")` → str; wrong codec = mojibake

INTERVIEW SAY THIS IN THE INTERVIEW

“What is REST? What makes an API RESTful?”

- **Resources** with URLs (**/orders/7**), manipulated with HTTP **methods** (GET, POST, PUT, PATCH, DELETE).
- **Stateless**: every request carries what’s needed (e.g. a token); the server keeps no session per client.
- Correct **status codes**, JSON representations, and cacheable GETs.
- Good practice: plural nouns, filtering with query params, pagination, versioning (**/v1/**).

> **Follow-up they ask next:** ”PUT vs PATCH? Which methods are idempotent?” (PUT replaces, PATCH changes some fields; GET, PUT and DELETE are idempotent, POST is not).

INTERVIEW SAY THIS IN THE INTERVIEW

“How do you prevent SQL injection? What is a transaction?”

- Never concatenate user input into SQL. Use **parameterised queries** or an **ORM**.
- Also: least-privilege DB users and input validation at the edges.
- A **transaction** groups statements so they all succeed or all roll back.
- **ACID**: Atomic, Consistent, Isolated, Durable.

```
cur.execute(
    "SELECT * FROM users WHERE email = %s",
    (email,))
```

> **Follow-up they ask next:** ”INNER vs LEFT JOIN? What does GROUP BY do? What is an index?” (see Module 23 and the Top 100 page).

RECAP REMEMBER THESE

Module 18 in 6 lines

1. **pathlib**, **csv.DictReader/Writer**, **json**, always **encoding="utf-8"**.
2. HTTP clients: **timeout**, **raise_for_status()**, retries for idempotent calls.
3. Know methods, status codes and **idempotency**.
4. SQL: JOIN, GROUP BY, **parameters** (no injection), **transactions**.
5. Store time in **UTC**, display in the user's zone.
6. Never unpickle untrusted data; bytes ↔ str with **utf-8**.

PRACTICE NOW

Do exercises/ex18_notes_repository.py : a SQLite repository with safe queries.